App Building with Flask

Enrique Calderon

LIDSoL

Semester 2026-2

App Building Overview

Goal

Build a small python endpoint using Flask. The program follows a standard project structure, dependencies, environment-based configuration, and basic logging.

Expected Duration

90 minutes

Python and Flasks

- Python-based micro web framework.
- Easy to use, excellent for simple APIs and endpoints.
- Requires `flask` via pip, we will use a virtual environment to manage dependencies.

```
python3 -m venv venv
source venv/bin/activate
```

Project Structure

- Standard layout for a Python web app:

```
my_app/
  - app.py
  - requirements.txt
  - .env
```

Requirements file

This file lists the dependencies of the project. It allows for easy installation and version control. Try to always pin versions to ensure reproducibility. For example:

```
Flask==3.1.3
gunicorn==25.1.0
python-dotenv==1.2.2
```

Environment Variables

Usually we have some kind of configuration that we don't want to hardcode in our codebase (like secrets, API keys, or environment-specific settings). We can use a `.env` file to store these variables and load them in our application using `python-dotenv`.

Example `.env` file:

```
SECRET_KEY=supersecretkey
```

Main Program

The app should read settings from environment variables (using python-dotenv) and just answer to a single endpoint (/) with a simple JSON message if the client sends the same secret key as an environment variable.

Code

```python
import os
import logging
from flask import Flask, request, jsonify
from dotenv import load_dotenv

logging.basicConfig(level=logging.INFO)

# Load variables from .env for local development.
load_dotenv()

app = Flask(__name__)
# Here we load the secret key from the environment
↪  variable, with a default fallback for development
app.config['SECRET_KEY'] = os.getenv('SECRET_KEY',
↪  'default-key').strip()
```

Writing the Endpoint

We will answer any POST request to the root endpoint (/) with a JSON message if the client sends the correct secret key in the request body.

Code

```python
@app.route('/', methods=['POST'])
def ping():
    data = request.get_json()
    request_secret = (data or {}).get('secret_key', '')
    if request_secret.strip() != app.config['SECRET_KEY']:
        logging.warning("Unauthorized access attempt with
↪  data: %s", data)
        return jsonify({'error': 'Unauthorized'}), 401
    logging.info("Successful access with data: %s", data)
    return jsonify({'message': 'Hello, World!'})

# Running the App on port 5000
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Running the App

To run the app, make sure you have your virtual environment activated and the dependencies installed:

```
pip install -r requirements.txt
```

Then you can start the Flask development server:

```
python app.py
```

Testing the Endpoint

You can test the endpoint using `curl` or a tool like `httpie`. Make sure to include the correct secret key in the request body.

```
curl -X POST http://127.0.0.1:5000/ \
    -H "Content-Type: application/json" \
    -d '{"secret_key": "supersecretkey"}'
```

or with httpie:

```
http POST http://127.0.0.1:5000/ secret_key=supersecretkey
```

Desired Outcome

If the correct secret key is provided, you should receive a JSON response with the message “Hello, World!”:

```
{
  "message": "Hello, World!"
}
```